

Lesson 8: Forms & User Input

Letting the user type

Apps are not just for reading -- users need to type things too. A login page needs an email and a password. A search bar needs a query. A feedback form needs a message.

In Flutter, the widget for text input is `TextField`. It shows a box the user can tap and type into.

A basic `TextField`

```
TextField(  
  decoration: const InputDecoration(  
    labelText: 'Your name',  
    hintText: 'Type your name here',  
    border: OutlineInputBorder(),  
  ),  
)
```

The decoration controls how the field looks:

- `labelText` -- a small label that sits above the text once the user starts typing.
- `hintText` -- faint helper text shown inside the empty field.
- `border` -- the outline around the field. `OutlineInputBorder()` gives the clean box shape.

Try it: the field looks plain, but tap it and the keyboard opens.

Reading what the user typed

A text field is useless if you cannot read what was typed. For that, you attach a `TextEditingController` -- an object that holds the current text of the field:

```
class NameForm extends StatefulWidget {  
  @override  
  State<NameForm> createState() => _NameFormState();  
}  
  
class _NameFormState extends State<NameForm> {  
  final nameController = TextEditingController();  
  
  @override  
  void dispose() {  
    nameController.dispose(); // clean up when done  
    super.dispose();  
  }  
}
```

```

@override
Widget build(BuildContext context) {
  return Column(
    children: [
      TextField(controller: nameController),
      ElevatedButton(
        onPressed: () {
          print(nameController.text); // the typed text
        },
        child: const Text('Submit'),
      ),
    ],
  );
}

```

Three things to notice:

- You create the controller once as a field of the State (not inside build, or it would reset on every rebuild).
- You pass it to the TextField with controller:.
- You read the text any time with controller.text.

The dispose() method cleans the controller up when the screen goes away. Forgetting it is a common beginner mistake -- always dispose what you create.

Checking the input: basic validation

Users make mistakes: empty fields, bad emails, short passwords. Validation means checking the input before you accept it.

The simplest check is plain Dart code when the button is pressed:

```

onPressed: () {
  final name = nameController.text.trim();
  if (name.isEmpty) {
    print('Please enter your name');
    return;
  }
  print('Hello, $name!');
},

```

trim() removes extra spaces at the start and end, so a field with only spaces still counts as empty. For a real app you would show the message on screen (for example, with a SnackBar) instead of printing it -- but the checking logic is the same.

Password fields

For passwords, you want dots instead of readable text. That is one property:

```
TextField(  
  obscureText: true,  
  decoration: const InputDecoration(  
    labelText: 'Password',  
    border: OutlineInputBorder(),  
  ),  
)
```

`obscureText: true` hides what is typed. Everything else works exactly like a normal field.

Key terms

- `TextField` -- the widget for text input.
- `InputDecoration` -- controls how a text field looks: labels, hints, borders.
- `TextEditingController` -- an object that holds and gives access to a field's text.
- `controller.text` -- the current text inside the field.
- `dispose()` -- cleans up the controller when the widget is removed; always call it.
- `Validation` -- checking user input for mistakes before accepting it.
- `obscureText` -- hides typed text with dots, used for passwords.